

Versionskontrolle mit Git & Databricks Git-Ordern

Section 5 · Implementing CI-CD · Quelle: Kurs 4, M02 Kapitel 14–15, M03 Kapitel 1

Ohne zentrale Versionskontrolle entstehen schnell Entwicklungs-Silos: Mehrere Personen arbeiten an Kopien desselben Notebooks, Änderungen lassen sich nicht mehr nachvollziehen, und es ist unklar, welche Version tatsächlich in Produktion läuft. Git löst dieses Problem durch nachvollziehbare Versionierung, parallele Entwicklung über Branches und einen zentralen Ort (das Remote-Repository, z. B. auf GitHub), auf den sich alle Beteiligten beziehen.

1. Branching-Strategie: Gitflow

Eine verbreitete Strategie ist **Gitflow**: Ein stabiler `main`-Branch spiegelt immer den Produktionsstand wider. Neue Funktionen werden in eigenen *Feature-Banches* entwickelt, über Pull Requests geprüft und dann in einen Integrations-Branch (oft `develop`) gemerged; Release- und Hotfix-Banches ermöglichen es, eine Version für die Veröffentlichung vorzubereiten bzw. dringende Produktionsfehler zu beheben, ohne laufende Feature-Entwicklung zu stören.

2. Databricks Git-Ordner (Git Folders)

Databricks integriert Git direkt in den Workspace über sogenannte **Git-Ordner** (früher „Repos“): Ein Git-Ordner ist ein Workspace-Verzeichnis, das an ein Remote-Repository gekoppelt ist und Standard-Git-Operationen (Pull, Commit, Push, Branch wechseln) über die Databricks-Oberfläche erlaubt. Typische Arbeitsweise:

- **Admin-Workflow:** Ein Workspace-Administrator legt zentrale Git-Ordner für Produktions-/Test-/Staging-Banches unter einem gemeinsamen Pfad an.
- **Nutzer-Workflow:** Jede Person klonet das Repository in einen eigenen Git-Ordner unter ihrem Benutzerverzeichnis, arbeitet dort auf einem eigenen Branch und pusht Änderungen ins Remote-Repository.
- **Merge-Workflow:** Nach Review und Merge eines Pull Requests wird der aktualisierte Stand automatisiert (z. B. per CI/CD-Pipeline) in die zentralen Produktions-Git-Ordner übernommen.

3. Authentifizierung mit einem GitHub Personal Access Token (PAT)

Damit Databricks im Namen einer Person auf ein privates GitHub-Repository zugreifen darf, wird in den Nutzereinstellungen ein **Personal Access Token** (PAT) hinterlegt. Dieses Token wird beim Verknüpfen des Git-Ordners mit dem Repository hinterlegt und ersetzt die klassische Passwort-Authentifizierung.

4. Automatisierung über die Repos-API

Für CI/CD-Automatisierung stellt Databricks eine **REST-API für Git-Ordner** bereit: Damit lässt sich z. B. nach einem erfolgreichen Merge eines Pull Requests automatisiert der entsprechende Produktions-Git-Ordner im Workspace auf den neuesten Commit aktualisieren – ganz ohne manuellen Eingriff einer Person im Workspace.

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks beschreibt für CI/CD mit Git-Ordnern konkret drei Abläufe: einen *Admin-Flow* (zentrale Produktions-/Test-/Staging-Ordner anlegen), einen *Nutzer-Flow* (persönlicher Git-Ordner unter `/Workspace/Users/<email>/` mit eigenem Branch) und einen *Merge-Flow* (nach Merge eines Pull Requests automatisiertes Update der Produktions-Git-Ordner über die Repos-API). Diese Struktur deckt sich mit dem im Kurs vermittelten Modell.

Quelle: [CI/CD with Databricks Git folders – Databricks-Dokumentation](#)